EvoRank: LLM-Guided Evolution of Multi-Objective Learning-to-Rank Pipelines

An evolutionary agent that builds new rerankers, and knows when an LLM edit will pay off.

Rayhan Patel[1,†] Shabaz Patel[2,†]

[1]University of Maryland, College Park [2]Independent Researcher [†]equal contribution

GenAIECommerce'26: The Third Workshop on Agentic and Generative AI for E-Commerce, RecSys 2026, Minneapolis, MN, USA

code + all results

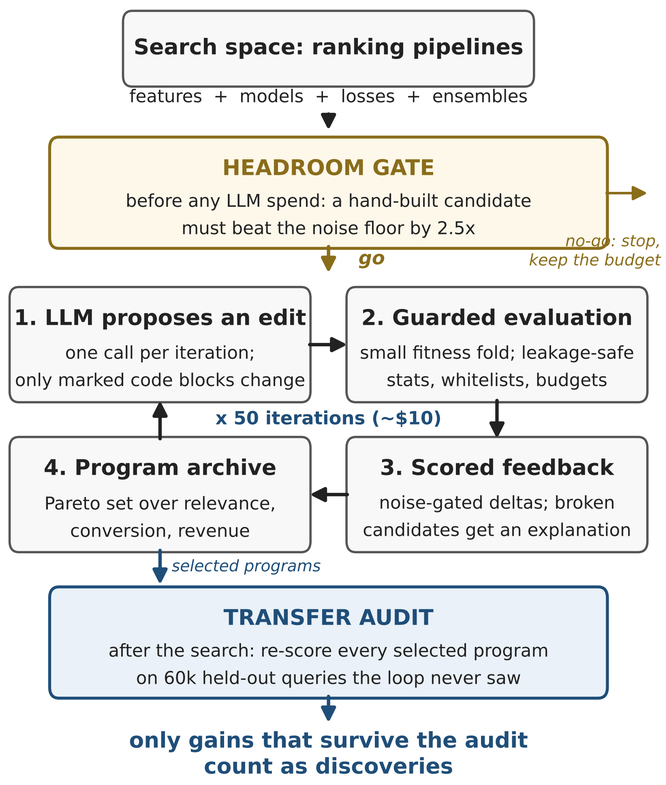
An Evolutionary Agent for Learning-to-Rank

EvoRank is an open evolutionary agent that engineers rerankers end to end. Each iteration, an LLM edits a candidate program: feature construction, model and loss selection, and ensembling. A guarded evaluator scores the edit on three competing business objectives, and a Pareto archive keeps the best trade-offs. In 50 iterations and about $10, the agent produces interpretable pipelines that beat a tuned LambdaMART on held-out data.

Contributions.

- **EvoRank**: an open evolutionary agent that discovers multi-objective rerankers as readable code, with seeds, logs, and audits.
- **Headroom gate**: decides *before any LLM spend* whether proposing edits in a search space can pay off, by comparing achievable effect sizes to evaluation noise.
- **Transfer audit**: only gains that survive 60k held-out queries count as discoveries.

System and Audit Methodology



The full audit stack, applied identically to every experiment: paired query bootstrap on every comparison; an equal-budget random-search control; equal Optuna tuning on both sides; the transfer audit; three-objective hypervolume; and component-removal tests on every discovered program.

Task. Expedia hotel search (ICDM 2013): 9.9M impressions, 399k queries, split 70/15/15 by query. Three objectives that genuinely compete, computed from real behavior: relevance (NDCG@10), conversion (booking NDCG@10), and revenue (pooled dollar-weighted capture).

Campaign 1: Why the Gate Is Needed

We first pointed the agent at a space with no headroom: the training objective alone, the LambdaMART gradient. On its selection fold it improved steadily. The transfer audit told a different story.

On 60k held-out queries: the best evolved objective beats its own starting program by +0.0002 NDCG@10 ($p = 0.76$); blind random search at the same budget transfers as well or better ($p = 0.03$); and every method sits below the tuned LambdaMART baseline ($p < 0.001$).

Why, measured: one component's true effect is 0.0005 to 0.008 NDCG@10, while the fold's measurement noise is ±0.007 to 0.009. When the effect is smaller than the ruler, picking the best score is picking luck.

Seeded knowledge made the search faster, not truer; richer feedback changed nothing that transferred. The gate predicts this outcome in advance, for the cost of one hand-built candidate, and saves the entire spend.

Campaign 2: The Agent Discovers Rerankers

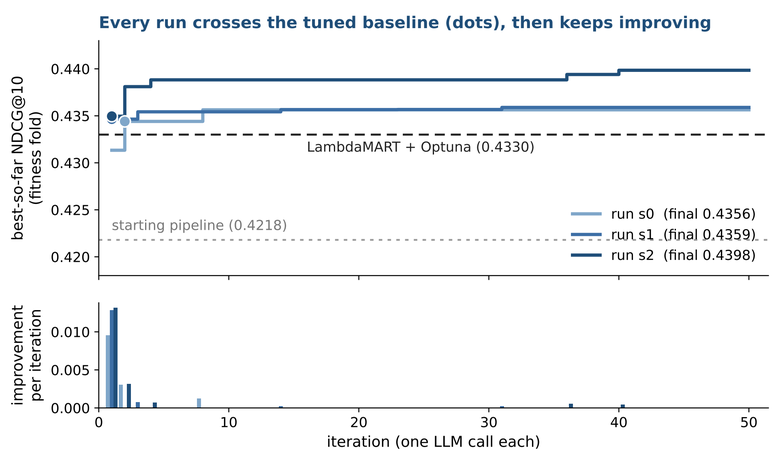
Same agent, aimed where the gate approves: the full pipeline space, behind leakage-safe helpers (+0.013 headroom, over 2.5× the noise floor). Three runs, 50 iterations:

Method (fitness-fold trained)	Val NDCG@10	**Test NDCG@10**	Test book	Test revenue
Starting pipeline	0.4218	0.4222	0.4443	0.4105
LambdaMART + Optuna	0.4330	0.4296	0.4527	0.4231
Pipeline s0	0.4356	0.4316	0.4545	**0.4257**
Pipeline s1	0.4359	0.4322	0.4553	0.4215
Pipeline s2	0.4398	**0.4352**	**0.4582**	0.4239

Transfer audit on 59,902 held-out queries; bold = column best.

Every run beats the tuned baseline on held-out data; the best wins by +0.0057 NDCG@10 (95% CI $[0.0041, 0.0073]$, $p < 0.0001$), with revenue matching.

Convergence Analysis



Best-so-far fitness NDCG@10 per iteration (top) and each iteration's increment (bottom). All three runs pass the tuned baseline within two iterations and keep finding increments to iteration 40; the table above shows what survived the audit.

Full-Scale Evaluation

280k train / 60k test queries	NDCG@10	NDCG@38
Tuned baseline (small-fold tuning)	0.4349	0.4983
Tuned baseline (full-scale tuning)	0.4544	0.5132
Pipeline s1 (small-fold tuning)	0.4490	0.5080
Pipeline s1 (baseline's tuned settings)	**0.4619**	**0.5185**

Features alone: 0.4587 (+0.0043); the ensemble adds +0.0032. Full-scale bootstrap: +0.0075 NDCG@10, $p < 0.0001$.

Kaggle late submission (one shot, hidden test set): discovered pipeline **0.5196** private NDCG@38 → would rank **20 of 340 (top 6%)** in the 2013 challenge; tuned baseline 0.5140 (30th). Ten leaderboard places from the discovered structure alone.

The Discovered Pipeline

All three runs converged on the same simple recipe, as readable code:

Features (about 25, deliberately few):

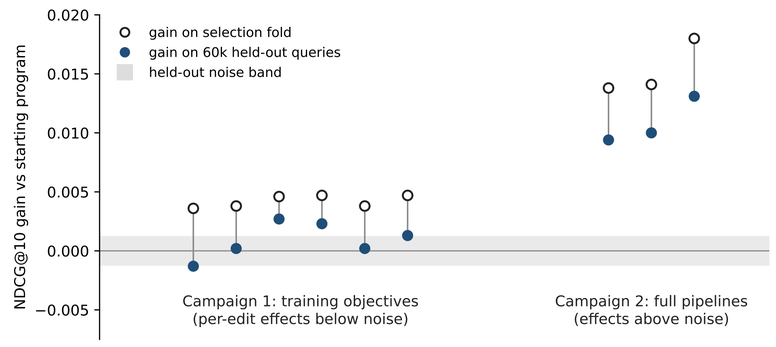
- rank price, stars, review and location scores *within each search*, so hotels compete against what they were shown with
- count how often each hotel and destination appears (popularity)

Model: three different learners, blended per query:

```
0.50 x XGBoost (rank:ndcg) + 0.35 x LightGBM (lambdarank) + 0.15 x ExtraTrees
```

This is the structure the 2013 challenge winners engineered by hand. The loop found it on its own in 50 iterations, and every piece is inspectable code, not weights.

Transfer Audit Results



The whole study in one picture: per-run NDCG@10 gain over the starting program, on the selection fold (open) versus 60k held-out queries (filled). Campaign 1's apparent gains collapse into the noise band after selection; campaign 2's survive.

Open everything: code, configs, seeds, logs, discovered programs, audit scripts at `github.com/shabazpatel/evorank`. Limitations: one dataset (the only public LTR set with conversion and revenue labels); three runs per condition; revenue is a proxy; audited for transfer, not online effects.

A Reusable Procedure for LLM-Driven Ranking Discovery

1. Measure noise
Paired query bootstrap on the evaluation fold you can afford: that half-width is your noise floor.

2. Hand-build one candidate
One reference candidate from your domain's literature, before any LLM spend.

3. Gate the space
Require its gain ≥ 2.5× the noise floor. No headroom, no loop: keep your budget.

4. Run guarded
Leakage-safe statistics, whitelists, budgets, and rejection messages the model learns from.

5. Audit, then believe
A held-out transfer audit, never the loop's own scores, decides what counts.

Contact: rayhanbp@umd.edu • shabaz.b.patel@gmail.com

Paper + proceedings: genai-ecommerce.github.io/GenAIECommerce2026